Student No._____ **Name:**_____

특별한 언급이 없는 한, 32bit x86 프로세서의 protected mode 에서 MASM 을 사용함을 가정함

1.1 [5pt] 이전의 컴퓨터들과 대비되는 폰 노이만 컴퓨터 구조의 주된 특징은 무엇인가?

1.2 [5pt] 이로 인해 폰 노이만 컴퓨터 구조가 갖게 되는 주된 병목지점(bottle neck)은 무엇인가?

2. [10pt] 당신은 새로운 프로세서를 디자인하고 있다. 이 프로세서에서 사용될 '수의 체계'로써, 부호를 갖는 2 진수를 다음과 같이 심플하게 설계하고자 한다.

- ✓ MSB 는 부호를 의미하는 비트로 0 일 경우 양수, 1 일 경우 음수를 의미함
- ✓ MSB 를 제외한 나머지 비트들은 일반적인 2 진수 표기법을 사용하며, 수의 절대값을 의미함
- ✓ 따라서 어떤 양의 2 진수 표현에서 MSB 를 1 로 바꿀 경우, 절대값은 같고 부호만 바뀜
  예) 8 비트의 경우, 5 는 00000101, -5 는 10000101 로 표현함

현대의 컴퓨터 시스템들과 비교했을 때, 당신이 설계한 2 진수 체계를 사용하는 프로세서의 단점은 무엇인가?

3. [5pt] 다음은 오늘날 가장 일반적인 컴퓨터 시스템의 구조에 대한 설명이다. **옳지 <u>않은</u>** 설명을 **<u>모두</u>** 고르시오. **답:**_____

① CPU 는 레지스터, 클럭, 제어 장치, 산술 논리 장치, 주기억장치, 보조기억장치 등으로 구성된다.

② 주기억장치는 프로그램의 명령어를, 보조기억장치는 데이터를 저장한다.

③ 메모리 접근 속도를 향상시키기 위해 주기억장치와 보조기억장치 사이에 캐쉬를 두곤 한다.

④ Bus 는 컴퓨터의 구성요소들간 데이터 또는 명령어 전송을 담당한다.

⑤ CPU 클럭의 한 사이클은 하나의 명령어(또는 데이터)가 메모리로부터 CPU 로 전송되는 시간을 의미한다.

4. [10pt] 다음은 명령어 실행 사이클에서 CPU 가 수행하는 핵심적인 세 단계에 대한 설명이다. 주어진 괄호를 채우시오. (괄호 하나당 하나의 단어만 필요하며, 단어들은 서로 중복될 수 있음)

- ✓ **1 단계**: (①)(으)로부터 (②)을/를 가져온다.
- ✓ **2 단계**: (③)을/를 (④)한다.
- ✓ **3 단계**: (⑤)을/를 (⑥)한다.

5. [10pt] x86 프로세서의 real address 모드에서는 address bus 와 data bus 의 대역폭이 다르기 때문에 segment 와 offset 을 이용해 메모리에 접근한다. 이러한 맥락에서 real address 모드에서의 주소 공간의 최대 크기를 계산하시오. (계산 과정 필요)

6.1 [6pt] 다음 어셈블리어 코드의 마지막 구문이 실행된 후, Carry Flag (CF), Overflow Flag (OF), Sign Flag (SF) 세팅 여부를 답하시오.

```
MOV AX,  2A8h
MOV BX, 13C0h
ADD AL, BL
```

CF	OF	SF

6.2 [9pt] 6.1 에서 CPU 가 그것을 어떻게 탐지하는지 자세한 계산과정과 함께 설명하시오.

1) CF

2) OF

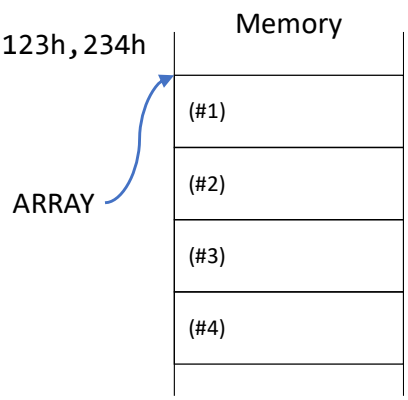
3) SF

Student No._____ **Name:**_____

7. 다음 주어진 어셈블리어 프로그램들에 대한 물음에 답하시오.

(1) [10pt] MASM 으로 작성된 어느 프로그램이 아래와 같은 data segment 를 가지고 있다. 메모리를 표현한 오른쪽 그림상에서 ARRAY 의 위치가 화살표와 같을 때, #1~4 에 들어갈 16 진수의 값을 그림 속에 채워넣으시오.
(그림 상에서 위쪽이 더 작은 주소를 의미함.)

```
.data
ARRAY WORD 123h,234h
```



(2) [10pt] 다음 MASM 프로그램의 마지막 명령어를 실행 한 직후의 이래 flag bit 들의 값을 예상하시오.

```
.data
ARRAY SWORD -1,-2,-3,-4,-5

.code
MOV AL, BYTE PTR [ARRAY] + 2
ADD AL, 2
```

Flag bit	값
CF	
OF	
SF	
ZF	

(3) [10pt] 다음의 C/C++ 프로그램을 어셈블리어 프로그램으로 변환하도록 빈칸을 채우시오.

```
char array[] = { 10, 20, 30, 40, 50 };
char sum = 0;

for (int i = 0; i < sizeof array; ++i)
{
    sum += array[i];
}
```

```
    .data
    ARRAY BYTE 10, 20, 30, 40, 50
    SUM   BYTE 0

    .code
(#1)
    MOV ESI, 0
L1:
    ADD SUM, ARRAY[ESI]
    INC ESI
(#2)
```

(#1)	
(#2)	

(4) [10pt] 다음 프로그램에서 박스 속의 명령이 실행된 직후 ESP 가 가리키고 있는 값을 16 진수로 쓰시오.

```
.code
main PROC
MOV eax, -1
MOV  ax, 128
PUSH eax
CALL mysub
ADD  eax, 64
INVOKE ExitProcess, 0
main ENDP

mysub PROC
ADD eax, 32
ret
mysub ENDP
END main
```

ESP 가 가리키는 값	